

Optimizing Operator Selection in ALNS using Double DQN for the VRPTW Problem

Huynh Nhat Huy*, Nguyen Nhat Huy*, Nguyen Thi Bao Tran*

Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam

Advisor: Dr. Ho Thi Linh

Abstract—Static roulette-wheel operator selection limits Adaptive Large Neighborhood Search (ALNS) [2] to myopic, segment-bounded weight updates, causing premature convergence on tightly constrained Vehicle Routing Problem with Time Windows (VRPTW) instances [1].

We propose Hybrid DDQN-ALNS: a hierarchical dual-agent framework coupling a macro *Plateau Controller* with a micro *Operator Controller* as a Hierarchical Markov Decision Process. Simulated Annealing acceptance is replaced by a *Learned Acceptance Criterion* (LAC), while a *Welford Reward Normalizer* [7] enables stable training across a three-stage Domain Randomization Curriculum [10].

On 55 Solomon 100-customer and 6 Gehring–Homberger 200-customer instances, Hybrid DDQN-ALNS reduces mean vehicle inflation to +0.182 above BKS—a 62% improvement over ALNS-Base and 90% over OR-Tools [15]. NV-filtered travel distance improves by 63% on wide-horizon families, driven by state-conditioned sequence polishing. All results use zero-shot transfer from synthetic training data.

Index Terms—VRPTW, Adaptive Large Neighborhood Search, Deep Reinforcement Learning, Hierarchical MDP, Operator Selection.

I. INTRODUCTION

The VRPTW lexicographically minimises fleet size then travel distance subject to per-customer time windows $[e_i, l_i]$ —an NP-hard problem [1] whose search space grows exponentially with customer count.

ALNS [2]–[4] is a leading metaheuristic for VRPTW; however, its roulette-wheel operator selection has three structural weaknesses: (i) myopic, reward-dominated operator ranking; (ii) premature plateau convergence; and (iii) static SA acceptance that cannot adapt to route topology.

End-to-end neural approaches—Pointer Networks [12], attention models [13], and RL policies [14]—demonstrate strong performance on unconstrained VRP variants but do not enforce VRPTW’s lexicographic fleet-size discipline and offer limited interpretability of the learned search policy. Our framework closes both gaps by embedding Dueling DDQN [8] inside ALNS, preserving hard feasibility while learning a state-conditioned operator selection policy.

Contributions:

- **Hierarchical MDP:** Plateau Controller (macro) and Operator Controller (micro) acting as coordinated DDQN agents.

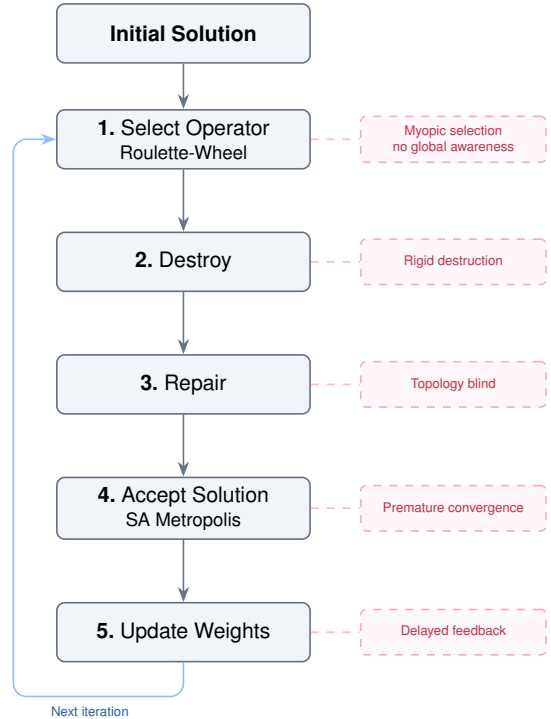


Fig. 1. Classical ALNS [2] and its structural limitations.

- **Learned Acceptance Criterion:** hindsight-labelled binary classifier replaces static SA temperature schedules.
- **Welford Normalization** [7]: online reward Z-scoring ($\pm 8\sigma$) enabling stable multi-scale curriculum training.
- **Empirical validation:** 55 Solomon + 6 Gehring–Homberger instances with NV-filtered distance reporting, family-level decomposition, and Wilcoxon significance tests.

II. MATHEMATICAL BACKGROUND

The VRPTW is defined on graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ with $\mathcal{V} = \mathcal{N} \cup \{0, n+1\}$. Customer $i \in \mathcal{N}$ has demand q_i , service time s_i , and window $[e_i, l_i]$. A fleet of K homogeneous vehicles with capacity Q serves all customers.

A. Formulation

Let $x_{ij}^k \in \{0, 1\}$ denote arc traversal and $w_i^k \geq 0$ service start time. The lexicographic objective [5] prioritises fleet size:

$$\min \left(\mathcal{M} \sum_{k,j} x_{0j}^k + \sum_{k,(i,j)} c_{ij} x_{ij}^k \right), \quad \mathcal{M} \gg 1 \quad (1)$$

subject to visit-once (2), flow conservation (3), and capacity (4):

$$\sum_k \sum_{j \neq i} x_{ij}^k = 1, \quad \forall i \in \mathcal{N} \quad (2)$$

$$\sum_i x_{ih}^k - \sum_j x_{hj}^k = 0, \quad \forall h, k \quad (3)$$

$$\sum_i q_i \sum_j x_{ij}^k \leq Q, \quad \forall k \quad (4)$$

Time feasibility is enforced via Big- $\mathcal{M}$:

$$w_i^k + s_i + t_{ij} - \mathcal{M}(1 - x_{ij}^k) \leq w_j^k \quad (5)$$

$$e_i \leq w_i^k \leq l_i, \quad \forall i \in \mathcal{V} \quad (6)$$

Figure 2 illustrates time propagation: early arrival incurs mandatory waiting; late arrival is infeasible.

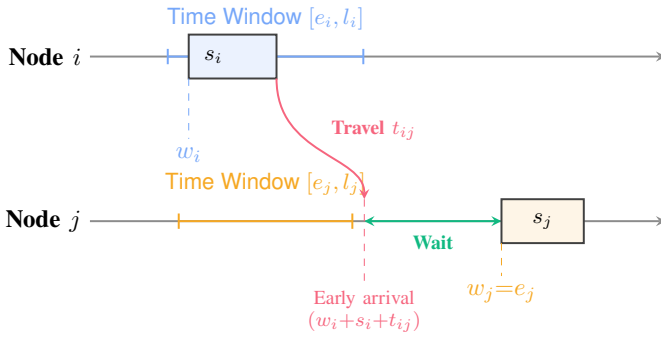


Fig. 2. Time propagation: early arrival waits; late arrival is infeasible [5].

III. PROPOSED METHODOLOGY: HYBRID DDQN-ALNS

Hybrid DDQN-ALNS preserves the feasibility guarantees of ALNS while replacing its static roulette-wheel operator selection with a hierarchical reinforcement learning stack. The search is modeled as a Hierarchical Markov Decision Process (HMDP) with two coordinated agents: a *Plateau Controller* (macro) that selects among six search regimes, and an *Operator Controller* (micro) that picks destroy–repair operator pairs conditioned on the active regime. Figure 3 illustrates the architecture.

A. Hierarchical MDP Formulation

The HMDP operates at two temporal granularities. Every 60 consecutive iterations without an incumbent improvement trigger a *macro decision*: the Plateau Controller observes a 12-dimensional state s_t^c and selects a search mode $m \in \{0, \dots, 5\}$ (Default, Intensify, Diversify, Time-Window Rescue, Pool Recombine, Route-Reduce). At every iteration, the Operator Controller observes a 15-dimensional state s_t^o —augmented with the active mode m as a goal signal—and selects a destroy–repair pair $a_t \in \{0, \dots, 54\}$. Table I summarizes both state representations.

TABLE I

STATE VECTOR COMPOSITIONS FOR THE TWO DDQN CONTROLLERS.

Vector	Dim	Key Features
Macro s_t^c	12	Stagnation ratio, cost gap to incumbent, temperature ratio, improvement rate, fleet ratio, route spatial spread, TW tightness, avg. slack, avg. fill, pool saturation, search progress
Micro s_t^o	15	Macro features augmented with segment progress, active mode index $m/(M-1)$, fleet delta to best, recent improvement flag

Both controllers share a lexicographic reward potential that dynamically balances fleet reduction and distance minimization:

$$\Phi(x) = -\lambda_{fp} \beta_{nv} \Delta_{NV}^{\text{norm}} - (1 - \lambda_{fp}) \beta_{td} \text{Gap}_{TD} \% \quad (7)$$

where $\lambda_{fp} = \sigma(-8(NV_x - NV^*)/NV_{\text{init}})$ is a sigmoid fleet-pressure gate, $\Delta_{NV}^{\text{norm}}$ is the normalized excess fleet size, and $\text{Gap}_{TD} \%$ is the clipped travel-distance gap to the incumbent. Reward shaping via $F_t = \gamma \Phi(x_{t+1}) - \Phi(x_t)$ guarantees convergence toward lexicographically superior solutions.

B. Operator Selection and Acceptance

Plateau Controller. The macro agent is a Dueling DDQN [8] (hidden dim. 128) evaluated at every stagnation trigger. Each of the six modes configures a distinct operator weight bias and temperature schedule to steer the search between intensification and diversification. The segment reward evaluates search quality over $S=100$ iterations:

$$R_{\text{seg}} = \beta_{\text{seg}} [R_{\text{base}} + \lambda_{fp} R_{nv} + (1 - \lambda_{fp}) R_{td}] + \gamma_c \Phi(x_{t+S}) - \Phi(x_t) \quad (8)$$

where R_{base} , R_{nv} , and R_{td} respectively penalise operational cost, reward vehicle reduction, and reward distance improvement.

Operator Controller. The micro agent is a Dueling DDQN (hidden dim. 256) selecting from 11 destroy $\times$ 5 repair = 55 operator pairs at every iteration. Operator scoring blends Q-values with Thompson sampling [11] bandit means and mode-conditioned priors:

$$\text{Score}(a) = Q(s, a) + \alpha_b \mu_t^{\text{bandit}}(a) + \text{UCB}(a) + \alpha_p \ln P(a | m) \quad (9)$$

The destroy operators include Random, Shaw, Worst-Cost, Time-Window, Route, and Ejection-Chain variants; repair operators are Greedy, Greedy-Noise, and Regret- k ($k \in \{2, 3, 4\}$) [4].

Learned Acceptance Criterion (LAC). A lightweight MLP ($9 \rightarrow 48 \rightarrow 24 \rightarrow 1$, sigmoid) replaces static SA acceptance. Nine features characterizing the candidate solution—including cost delta, temperature ratio, fleet change, and Metropolis probability—predict whether acceptance leads to an incumbent

Domain Randomization Curriculum — Adaptive Phase Controller

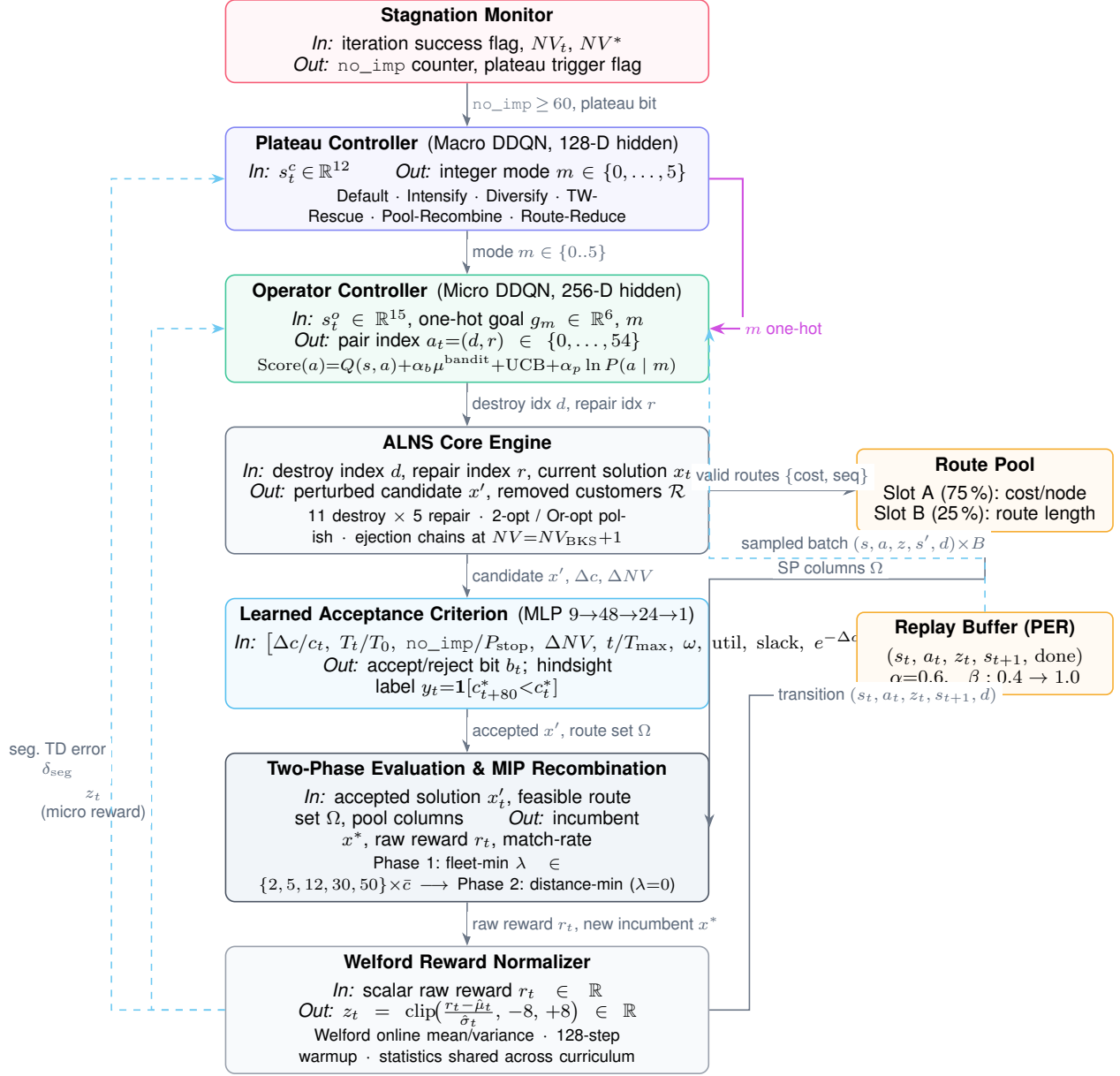


Fig. 3. Hybrid DDQN-ALNS architecture. Each block shows its In/Out interface. Solid arrows: forward data flow. Dashed arrows: reward feedback loops. Magenta arrow: goal-conditioning mode vector.

improvement within horizon $H=80$. Labels are assigned via hindsight:

$$y_t = \begin{cases} 1 & \text{if } c_{t+H}^* < c_t^* \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

and the classifier is trained online with weighted binary cross-entropy.

Search polishing. Intra-route 2-opt and Or-opt [4] refine each route until convergence ($\Delta c < 10^{-9}$). At $NV = NV_{\text{BKS}} + 1$, hard-mode ejection chains widen beam search (16→32) and deepen the cascade (6→10) to eliminate near-

empty vehicles.

C. Training and Infrastructure

Domain Randomization Curriculum. Following Tobin et al. [10], training uses only synthetic graphs across three phases:

Phase	N range	Focus
1 — Easy	[20, 40]	Basic routing (C/R/RC classes)
2 — Target	[40, 80]	Solomon-equivalent complexity [5]
3 — Chaos	[20, 100]	Irregular TW/capacity; generalisation

TABLE II
NV_{DIFF} ON SOLOMON. LOWER IS BETTER; 0.000=BKS.

Subset	ALNS [2]	H-Fixed	H-Rule	H-DDQN	OR-Tools [15]
Clustered ($N = 17$)	0.000	0.000	0.000	0.000	0.000
Short horizon ($N = 20$)	1.100	0.575	0.450	0.450	2.350
Wide horizon ($N = 18$)	0.250	0.167	0.056	0.056	3.111
Overall mean	0.482	0.264	0.182	0.182	1.873

Phase advancement requires a 50-episode match-rate threshold, preventing premature graduation. After $N_{\text{epochs}}=20$ cycles of $B=15$ graphs each, frozen weights are applied zero-shot to Solomon [5] and Gehring–Hömler [6] benchmarks without fine-tuning.

Reward normalization. Raw rewards are Z-scored online using the Welford algorithm [7]:

$$z_t = \text{clip}\left(\frac{r_t - \hat{\mu}_t}{\hat{\sigma}_t}, -8, +8\right) \quad (11)$$

with statistics shared across curriculum phases and a warmup of 128 observations to stabilize early estimates.

Experience replay and route storage. Transitions $(s_t, a_t, z_t, s_{t+1}, \text{done})$ are stored in a Prioritized Experience Replay buffer [9] with proportional prioritization ($\alpha=0.6$, β annealed $0.4 \rightarrow 1.0$). A dual-retention route pool maintains diverse columns for set-partitioning recombination: Slot A (75%, sorted by cost per customer) preserves locally efficient segments, while Slot B (25%, sorted by route length) retains fleet-reducing columns. When triggered, a two-phase MILP first minimizes fleet size via escalating vehicle penalties, then minimizes travel distance with zero penalty.

IV. EXPERIMENTAL RESULTS

A. Setup

Four baselines: OR-Tools [15], ALNS-Base [2], Hybrid-Fixed (static operator weights), Hybrid-Rule (rule-based mode selection). Solomon [5]: 5,000 iterations, 5 seeds, 55 instances (R211 excluded via hardware timeout). Gehring–Hömler [6]: 600 iterations, 6 representative instances.

Average Solomon runtimes: 28.5 s (ALNS-Base), 44.2 s (Hybrid-Rule), 56.4 s (Hybrid-DDQN). Neural inference adds 27.6% overhead over the static-rule baseline. Numba-JIT reduces 200-customer DDQN evaluation from 861.3 s to 60.3 s ($14\times$).

B. Vehicle Count Minimization

Table II reports $NV_{\text{diff}} = NV_{\text{algo}} - NV_{\text{BKS}}$. Hybrid-DDQN achieves +0.182 overall—62% below ALNS-Base and 90% below OR-Tools [15]. Wilcoxon signed-rank tests confirm significance against OR-Tools ($p = 5.932 \times 10^{-8}$, $\text{stat} = 0.0$) and ALNS-Base ($p = 1.314 \times 10^{-4}$, $\text{stat} = 0.0$).

C. Routing Distance with Fair-Subset Filtering

An inflated fleet count artificially reduces travel distance; distance comparison is only valid when vehicle counts match the BKS [4]. Two filters are reported: (a) *algo-specific*: each

TABLE III
NV-FILTERED TD GAP%. LOWER IS BETTER.

Subset	ALNS	H-Fixed	H-Rule	H-DDQN
Algo-specific	+5.250% ($N=32$)	+2.196% ($N=40$)	+2.012% ($N=45$)	+1.558% ($N=45$)
Fair intersection	+5.250%	+1.993%	+1.609%	+1.342%

TABLE IV
STRICT FAIR INTERSECTION ($N = 32$) BY SOLOMON FAMILY.

Category	ALNS	H-Fixed	H-Rule	H-DDQN
Clustered ($N = 17$)	+2.862%	+0.087%	+0.059%	+0.059%
Short horizon ($N = 2$)	+2.640%	+0.755%	+0.195%	+0.015%
Wide horizon ($N = 13$)	+8.775%	+4.677%	+3.852%	+3.223%
Overall ($N = 32$)	+5.250%	+1.993%	+1.609%	+1.342%

solver filtered independently; and (b) *strict fair intersection* ($N = 38$): all hybrids simultaneously at $NV \leq NV_{\text{BKS}}$.

Table IV disaggregates by family. The wide-horizon gap narrows from +8.775% (ALNS-Base) to +3.223% (Hybrid-DDQN), a 63% reduction attributable to convergent intra-route 2-opt and Or-opt polishing.

Ablation Wilcoxon tests confirm RL contributes beyond the rule baseline: DDQN vs. H-Rule ($p = 1.822 \times 10^{-5}$, $\text{stat} = 0.0$), DDQN vs. H-Fixed ($p = 2.275 \times 10^{-3}$, $\text{stat} = 119.0$). Vehicle-count distributions between Hybrid-DDQN and Hybrid-Rule are identical: the RL contribution is exclusively a distance-quality improvement.

D. Head-to-Head vs. OR-Tools

Because OR-Tools [15] inflates vehicle counts on 38 of 55 Solomon instances, symmetric distance comparison is only valid on the $N = 17$ clustered instances where OR-Tools achieves $NV \leq NV_{\text{BKS}}$. On this subset, Hybrid-DDQN reduces mean travel distance from 722.63 to 716.13 (-0.90%), with all 17 paired differences favourable (Wilcoxon $\text{stat} = 0.0$, $p = 0.068$). This does not reach $\alpha = 0.05$ given the small pool; we report it as a directional observation.

On non-clustered families, OR-Tools deploys +2.4 (short-horizon) and +3.0 (wide-horizon) extra vehicles on average ($p = 5.932 \times 10^{-8}$). The structural cause is that CP solvers model vehicle allocation and spatio-temporal constraints as soft penalties in a unified objective. Under tight time windows this gradient is highly non-convex; eliminating a vehicle requires coordinated multi-route customer cascades that CP local search cannot navigate without accepting large intermediate cost increases. Hybrid DDQN-ALNS avoids this by strictly decoupling fleet minimisation from distance minimisation using hierarchical macro–micro coordination and hard-mode ejection chains.

E. Gehring–Hömler Scalability

Table V shows 200-customer results [6]. Hybrid-DDQN matches BKS NV on all clustered instances, outperforms OR-Tools [15] by up to 8 vehicles on random families, and returns feasible solutions where OR-Tools fails (rc1_2_1). Negative

TD gaps in [†]-marked rows reflect vehicle over-allocation, not lexicographic improvement [4].

V. DISCUSSION

A. Learned State-Conditioned Neighborhood Selection

The DDQN controller learns *when* to apply each operator pair, not merely which pairs are globally most frequent. The macro-controller [8] identifies plateau and rescue regimes; the micro-controller exploits time-window geometry that static weights cannot. This is why Hybrid-DDQN equals Hybrid-Rule on vehicle counts (identical counts) while strictly dominating it on distance ($p = 1.822 \times 10^{-5}$, $stat = 0.0$).

Zero-shot transfer is decisive evidence of generalisation: weights are trained on synthetic domain-randomized graphs [10] and never tuned on Solomon or Gehring–Homerger data.

B. Limitations

RC101 Pareto alternative: Convergence to $NV = 15$ (vs. $NV_{BKS} = 14$) with total distance 2.6% below the literature reference [5] is a Pareto trade-off, not a failure.

Wide time-window gradient sparsity: R2 and RC2 families produce sparse reward surfaces because many service orderings share similar marginal costs. This explains the absent NV separation between Hybrid-DDQN and Hybrid-Fixed (identical vehicle counts), even though distance improvement remains clear (+3.223% vs. +4.677% on the wide-horizon fair subset).

Large-scale overhead: The $14 \times$ Numba-JIT speedup makes the 200-customer pilot practical, but graph-encoding optimizations [13] are required for 1000-customer deployment.

VI. CONCLUSION

Hybrid DDQN-ALNS replaces static roulette-wheel selection with a hierarchical dual-agent controller that learns state-conditioned operator policies across a domain-randomized curriculum. The architecture enforces strict lexicographic fleet-size discipline while achieving a 62% reduction in vehicle inflation over ALNS-Base, a 90% reduction over OR-Tools [15], and a 63% reduction in wide-horizon travel distance gap—all verified under zero-shot transfer to unseen benchmarks.

Future work will validate the ^{*} components (GNN encoder, N-step returns, HER relabelling, adaptive curriculum) via ablation on the Solomon and Gehring–Homerger suites, replace handcrafted state features with attention-based graph encodings [12], [13], and extend evaluation to Gehring–Homerger 1000-customer instances.

ACKNOWLEDGMENT

The authors thank Dr. Ho Thi Linh for her guidance throughout this work.

REFERENCES

- [1] G. B. Dantzig and J. H. Ramser, “The truck dispatching problem,” *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.
- [2] S. Ropke and D. Pisinger, “An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows,” *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.
- [3] P. Shaw, “Using constraint programming and local search methods to solve vehicle routing problems,” in *Principles and Practice of Constraint Programming (CP 1998)*, LNCS 1520, pp. 417–431, Springer, 1998.
- [4] D. Pisinger and S. Ropke, “A general heuristic for vehicle routing problems,” *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.
- [5] M. M. Solomon, “Algorithms for the vehicle routing and scheduling problems with time window constraints,” *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.
- [6] H. Gehring and J. Homberger, “A parallel two-phase metaheuristic for routing problems with time windows,” in *Proc. Asia-Pacific Decision Sciences Institute Conf.*, pp. 114–124, 1999.
- [7] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [8] Z. Wang *et al.*, “Dueling network architectures for deep reinforcement learning,” in *Proc. 33rd ICML*, vol. 48, pp. 1995–2003, 2016.
- [9] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *Int. Conf. Learning Representations (ICLR)*, 2016.
- [10] J. Tobin *et al.*, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proc. IEEE/RSJ IROS*, pp. 23–30, 2017.
- [11] W. R. Thompson, “On the likelihood that one unknown probability exceeds another in view of the evidence of two samples,” *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.
- [12] O. Vinyals, M. Fortunato, and N. Jaitly, “Pointer networks,” in *Advances in NeurIPS*, vol. 28, pp. 2692–2700, 2015.
- [13] W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!” in *Int. Conf. Learning Representations (ICLR)*, 2019.
- [14] M. Nazari, A. Oroojlooy, L. V. Snyder, and M. Takáč, “Reinforcement learning for solving the vehicle routing problem,” in *Advances in NeurIPS*, pp. 9861–9871, 2018.
- [15] Google LLC, “OR-Tools: Open source software for combinatorial optimization,” 2023. [Online]. Available: <https://developers.google.com/optimization>

TABLE V
GEHRING–HOMBERGER [6] 200-CUSTOMER RESULTS AT 600 ITERATIONS. FORMAT: NV/TD GAP%. † : $NV > NV_{\text{BKS}}$ — NEGATIVE GAPS REFLECT OVER-ALLOCATION.

Instance	BKS	ALNS [2]	H-Fixed	H-Rule	H-DDQN	OR-Tools [15]
<i>c1_2_1</i>	20/2704.57	20/ + 0.11%	20/ + 0.00%	20/ + 0.00%	20/ + 0.00%	21/ + 5.67%
<i>c2_2_1</i>	6/1931.44	6/ + 0.49%	6/ + 0.00%	6/ + 0.00%	6/ + 0.00%	7/ + 0.19%
<i>r1_2_1</i>	20/4795.12	20/ + 8.02%	20/ + 3.75%	20/ + 2.45%	20/ + 2.12%	26/ + 2.59%
<i>r2_2_1</i>	4/4483.00	5/ − 5.04% †	5/ − 7.69% †	5/ − 8.69% †	5/ − 8.89% †	12/ − 20.01% †
<i>rc1_2_1</i>	18/3603.00	19.4/ + 5.67% †	19/ − 0.52% †	19/ − 0.67% †	19/ − 0.88% †	21/ + 3.85%
<i>rc2_2_1</i>	6/3099.00	6.6/ + 3.00% †	6/ + 3.45%	6/ + 2.65%	6/ + 1.77%	11/ − 5.50% †